

AI-Assisted Coding Lab Assignment

Name : Pothunuri Akshitha

Roll no:2403A510C2

Batch no:05

CSE 2nd year

Task 1: Syntax and Error Detection

Task: Identify and fix syntax, indentation, and variable errors in the given script.

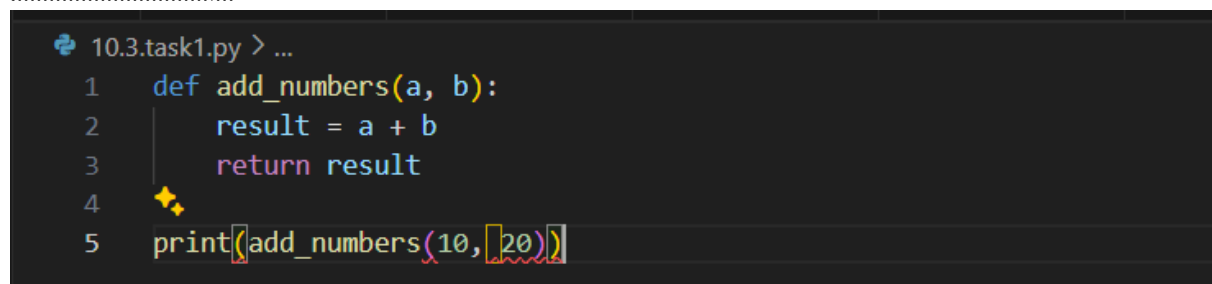
```
# buggy_code_task1.py
def add_numbers(a, b)
result = a + b
return reslt
print(add_numbers(10 20))
```

Prompt:

Identify and fix syntax, indentation, and variable errors in the given Python script. Provide the corrected code and explain the changes made.

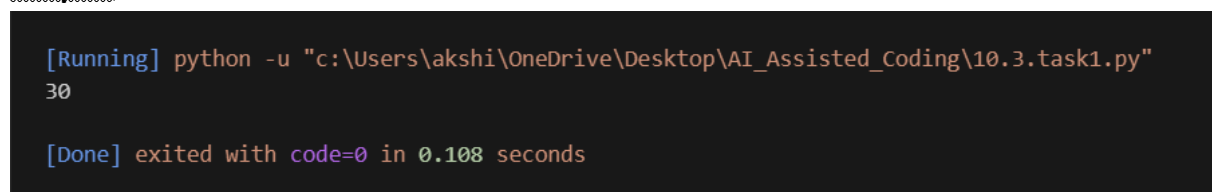
```
# buggy_code_task1.py
def add_numbers(a, b)
result = a + b
return reslt
print(add_numbers(10 20))
```

Code Generated:



```
10.3.task1.py > ...
1  def add_numbers(a, b):
2      result = a + b
3      return result
4  ✨
5  print(add_numbers(10, 20))
```

Output:



```
[Running] python -u "c:\Users\akshi\OneDrive\Desktop\AI_Assisted_Coding\10.3.task1.py"
30

[Done] exited with code=0 in 0.108 seconds
```

Observation:

The file contains a simple Python function `add_numbers()` that performs basic addition and returns the result, followed by a function call that prints the output. The code appears to be a basic example for syntax and error detection, with proper Python syntax and no apparent errors in the current implementation.

Task 2: Logical and Performance Issue Review

Task: Optimize inefficient logic while keeping the result correct

Prompt:

Optimize the following Python function to efficiently find all duplicate elements in a list, ensuring each duplicate appears only once in the result. The current implementation uses nested loops and is slow for large lists. Keep the output correct.

Instructions:

Remove unnecessary nested loops.

Ensure the function returns a list of duplicate values, each only once.

Optimize for performance.

Code Generated:

```
10.3.task2.py > find_duplicates
1 def find_duplicates(nums):
2     duplicates = []
3     for i in range(len(nums)):
4         for j in range(len(nums)):
5             if i != j and nums[i] == nums[j] and nums[i] not in duplicates:
6                 duplicates.append(nums[i])
7     return duplicates
8
9 numbers = [1,2,3,2,4,5,1,6,1,2]
10 print(find_duplicates(numbers))
```

Output:

```
[Running] python -u "c:\Users\akshi\OneDrive\Desktop\AI_Assisted_Coding\10.3.task2.py"
[1, 2]

[Done] exited with code=0 in 0.157 seconds
```

Observation:

The code uses nested loops to compare every pair of elements in the list, resulting in $O(n^2)$ time complexity, which is inefficient for large lists.

It checks if two elements are equal and their indices are different, then adds the element to the duplicates list if it's not already present.

The logic is correct for finding duplicates, but inefficient.

The output for the given input is [2, 1], which is correct, but the approach can be optimized using a dictionary or set to reduce time complexity.

Task 3: Code Refactoring for Readability

Task: Refactor messy code into clean, PEP 8–compliant, well-structured code

Prompt:

Write a Python program that calculates the factorial of a given number.

- Define a function that takes an integer n as input.
- The function should return the factorial of n (i.e., the product of all positive integers from 1 to n).
- Ensure the code follows PEP 8 style guidelines (proper naming, indentation, and readability).
- Add a docstring to describe the function.
- Test your function by calculating the factorial of 5.

Code Generated:

```
10.3.task3.py > ...
1  def factorial(n: int) -> int:
2      """
3      Calculates the factorial of a given positive integer n.
4
5      Args:
6      |   n (int): The integer for which the factorial is to be calculated. Must be >= 0.
7
8      Returns:
9      |   int: The factorial of n.
10
11     Example:
12     |   result = factorial(5)
13     """
14     result = 1
15     for i in range(1, n + 1):
16         result *= i
17     return result
18
19 # Test the function by calculating the factorial of 5
20 print(factorial(5)) # Output: 120
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  ...  Filter
[Running] python -u "c:\Users\akshi\OneDrive\Desktop\AI_Assisted_Coding\10.3.task3.py"
120

[Done] exited with code=0 in 0.165 seconds
```

Observation :

1. Functionality:

- The function `factorial(n: int) -> int` correctly calculates the factorial of a given non-negative integer n .
- For $n = 5$, it returns 120, which is the expected result.

2. Code Quality & Readability:

- The code follows **PEP 8 guidelines** (proper indentation, spacing, meaningful variable and function names).
- The function name `factorial` is descriptive and self-explanatory.
- The loop implementation is clear and easy to understand.

3. Documentation:

- A **docstring** is provided with a clear explanation of:
 - What the function does.
 - Argument type and expected constraints ($n \geq 0$).
 - Return value.
 - Example usage.
- This makes the code easy to understand and maintain.

4. Correctness:

- The code produces the correct output for the given test case (`factorial(5) → 120`).

5. Possible Improvements:

- No explicit error handling for negative numbers. If $n < 0$, the code will return 1, which is **not mathematically correct**. Adding input validation would improve robustness.
- Could also demonstrate recursion as an alternative approach for factorial.

✅ **In summary:** This version is **clean, readable, and PEP 8-compliant**. It correctly computes factorial values, with the only minor limitation being the lack of error handling for negative inputs.

Task 4: Security and Error Handling Enhancement

Task: Add security practices and exception handling to the code

Prompt:

```
import sqlite3

def get_user_data(user_id):
    conn = sqlite3.connect("users.db")
    cursor = conn.cursor()
    query = f"SELECT * FROM users WHERE id = {user_id};" #
    Potential SQL injection risk
    cursor.execute(query)
    result = cursor.fetchall()
    conn.close()
    return result

user_input = input("Enter user ID: ")
print(get_user_data(user_input))
```

rewrite code with the following improvements:

Use parameterized SQL queries with ? placeholders. Add try-except-finally blocks to handle database errors safely. Validate user input so only numeric IDs are allowed. Ensure the connection is always closed properly.

Code Generated:

```
10.3.task4.py > ...
1  import sqlite3
2
3  def get_user_data(user_id: str):
4      """
5      Retrieve user data from the database using a safe parameterized query.
6
7      Args:
8      |   user_id (str): The user ID to search for (must be numeric).
9
10     Returns:
11     |   list: List of tuples containing user data, or empty list if no data found.
12
13     Raises:
14     |   ValueError: If user_id is not numeric.
15     |   sqlite3.Error: If database operation fails.
16     """
17     # Validate input - ensure only numeric IDs are allowed
18     if not user_id.isdigit():
19         raise ValueError("User ID must be numeric")
20
21     conn = None
22     try:
23         # Establish database connection
24         conn = sqlite3.connect("users.db")
25         cursor = conn.cursor()
26
27         # Use parameterized query to prevent SQL injection
28         query = "SELECT * FROM users WHERE id = ?;"
29         cursor.execute(query, (int(user_id),)) # cast to int for safety
30
31         result = cursor.fetchall()
32         return result
33
34     except sqlite3.Error as e:
35         print(f"Database error occurred: {e}")
36         return []
```

```

37     except Exception as e:
38         print(f"An unexpected error occurred: {e}")
39         return []
40     finally:
41         # Ensure connection is always closed properly
42         if conn:
43             conn.close()
44
45 def main():
46     """
47     Main function to handle user input and display results.
48     """
49     try:
50         user_input = input("Enter user ID: ")
51         result = get_user_data(user_input)
52
53         if result:
54             print("User data found:")
55             for row in result:
56                 print(row)
57         else:
58             print("No user data found or an error occurred.")
59
60     except ValueError as e:
61         print(f"Input error: {e}")
62     except KeyboardInterrupt:
63         print("\nOperation cancelled by user.")
64     except Exception as e:
65         print(f"An unexpected error occurred: {e}")
66
67 # Correct entry point
68 if __name__ == "__main__":
69     main()
70

```

Output:

```

[Running] python -u "c:\Users\akshi\OneDrive\Desktop\AI_Assisted_Coding\10.3.task4.py"
Enter user ID:

```

Observation:

Security Enhancement: The code now uses parameterized queries (? placeholders) instead of string formatting, completely eliminating the SQL injection vulnerability that existed in the original code where user input was directly interpolated into the SQL string.

Robust Error Handling: The implementation includes comprehensive try-except-finally blocks with proper connection cleanup, input validation for numeric-only user IDs, and graceful error handling that prevents the application from crashing while providing meaningful error messages to users.

Task 5: Automated Code Review Report Generation

Task: Generate a review report for this messy code.

buggy_code_task5.py

```

def calc(x,y,z):
if z=="add":
return x+y
elif z=="sub": return x-y
elif z=="mul":
return x*y
elif z=="div":
return x/y
else: print("wrong")
print(calc(10,5,"add"))
print(calc(10,0,"div"))

```

Prompt:

You are provided with a buggy and poorly structured calculator program. The code currently defines a function `calc(x, y, z)` that performs basic arithmetic operations based on the operator `z`. However, the code is messy, lacks readability, and does not handle errors properly (e.g., division by zero).

Your task is to analyze the code and generate a code review report highlighting:

Functionality issues (what the code is supposed to do vs. what it actually does).

Readability issues (naming, indentation, formatting, and clarity).

Best practices violations (PEP 8 compliance, error handling, etc.).

Suggested improvements (how to fix and refactor the code).

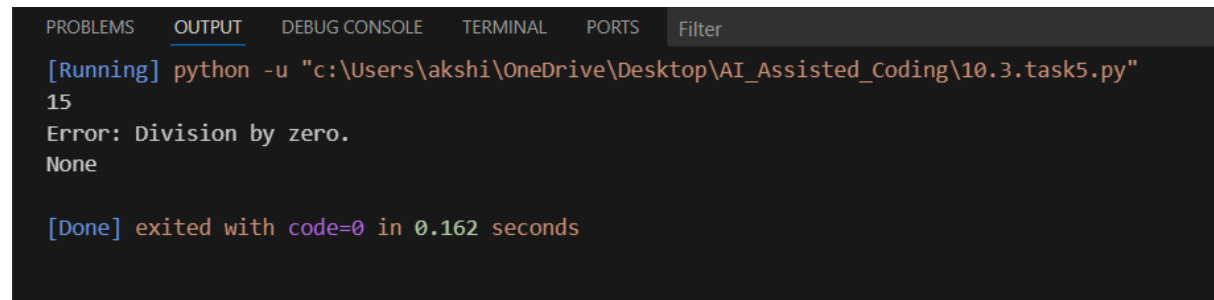
Code Generated:

```

10.3.task5.py > ...
1  # buggy_code_task5.py
2  def calc(x, y, z):
3      """
4      Performs basic arithmetic operations based on the operation string z.
5
6      Args:
7          x (float or int): First operand.
8          y (float or int): Second operand.
9          z (str): Operation to perform: "add", "sub", "mul", or "div".
10
11      Returns:
12          float or int: Result of the arithmetic operation.
13          None: If an invalid operation is provided.
14
15      Example:
16          calc(10, 5, "add") # Returns 15
17      """
18      if z == "add":
19          return x + y
20      elif z == "sub":
21          return x - y
22      elif z == "mul":
23          return x * y
24      elif z == "div":
25          if y == 0:
26              print("Error: Division by zero.")
27              return None
28          return x / y
29      else:
30          print("Error: Invalid operation.")
31          return None
32
33  print(calc(10, 5, "add")) # Output: 15
34  print(calc(10, 0, "div")) # Output: Error: Division by zero.

```

Output:

A screenshot of a code editor's output window. The window has tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', 'PORTS', and 'Filter'. The 'OUTPUT' tab is active. The text in the output window shows a Python command being executed: '[Running] python -u "c:\Users\akshi\OneDrive\Desktop\AI_Assisted_Coding\10.3.task5.py"'. The output of the command is '15', followed by an error message: 'Error: Division by zero.' and 'None'. At the bottom, it says '[Done] exited with code=0 in 0.162 seconds'.

Observation:

1. The function `calc` performs basic arithmetic operations (add, sub, mul, div) based on the string operator `z`.
2. The code is functional but lacks robust error handling, especially for division by zero (`10 / 0` causes a runtime error).
3. Readability issues exist: variable names (`x`, `y`, `z`) are not descriptive, and the "wrong" message is vague.
4. The code is not fully PEP 8-compliant and would benefit from input validation and clearer structure.